

Lab0 实验报告

一、思考题

1、objdump 参数和其他体系结构编译链接过程

- 在阅读 附录中的编译链接详解 以及本章内容后，尝试分别使用实验环境中的原生 x86 工具链（gcc、ld、readelf、objdump 等）和 MIPS 交叉编译工具链（带有 mips-linux-gnu- 前缀，如 mips-linux-gnu-gcc、mips-linux-gnu-ld），重复其中的编译和解析过程，观察相应的结果，并解释其中向 objdump 传入的参数含义。

#objdump --help 部分内容

```
-d, --disassemble      Display assembler contents of executable sections
-D, --disassemble-all  Display assembler contents of all sections
--disassemble=<sym>    Display assembler contents from <sym>
-S, --source            Intermix source code with disassembly
--source-comment[=<txt>] Prefix lines of source code with <txt>
```

可以看出 `-d` 选项代表仅显示 `.text` 段的反汇编代码，`-D` 选项代表显示所有段的反汇编代码，`-S` 选项代表同时显示反汇编代码和源代码

```
// adds.c
#include <stdio.h>

int main() {
    int a,b;
    scanf("%d %d", &a, &b);

    printf("%d", a+b);

    return 0;
}
```

```
gcc -c adds.c
objdump -d adds.o > test
```

test文件的内容

adds.o: 文件格式 elf64-x86-64

Disassembly of section .text:

```
0000000000000000 <main>:
 0:  f3 0f 1e fa      endbr64
 4:  55               push    %rbp
 5:  48 89 e5         mov     %rsp,%rbp
 8:  48 83 ec 10      sub     $0x10,%rsp
 c:  64 48 8b 04 25 28 00  mov     %fs:0x28,%rax
13:  00 00
15:  48 89 45 f8      mov     %rax,-0x8(%rbp)
19:  31 c0            xor     %eax,%eax
1b:  48 8d 55 f4      lea     -0xc(%rbp),%rdx
```

```

1f:  48 8d 45 f0          lea    -0x10(%rbp),%rax
23:  48 89 c6             mov    %rax,%rsi
26:  48 8d 05 00 00 00 00 lea    0x0(%rip),%rax      # 2d <main+0x2d>
2d:  48 89 c7             mov    %rax,%rdi
30:  b8 00 00 00 00      mov    $0x0,%eax
35:  e8 00 00 00 00      call   3a <main+0x3a>
3a:  8b 55 f0             mov    -0x10(%rbp),%edx
3d:  8b 45 f4             mov    -0xc(%rbp),%eax
40:  01 d0               add    %edx,%eax
42:  89 c6               mov    %eax,%esi
44:  48 8d 05 00 00 00 00 lea    0x0(%rip),%rax      # 4b <main+0x4b>
4b:  48 89 c7             mov    %rax,%rdi
4e:  b8 00 00 00 00      mov    $0x0,%eax
53:  e8 00 00 00 00      call   58 <main+0x58>
58:  b8 00 00 00 00      mov    $0x0,%eax
5d:  48 8b 55 f8          mov    -0x8(%rbp),%rdx
61:  64 48 2b 14 25 28 00 sub    %fs:0x28,%rdx
68:  00 00
6a:  74 05               je     71 <main+0x71>
6c:  e8 00 00 00 00      call   71 <main+0x71>
71:  c9                  leave
72:  c3                  ret

```

```

gcc -o adds.o adds.c
objdump -d adds.o > test

```

test文件的内容

adds.o: 文件格式 elf64-x86-64

Disassembly of section .init:

```

00000000000001000 <_init>:
   1000:    f3 0f 1e fa          endbr64
   1004:    48 83 ec 08          sub    $0x8,%rsp
   1008:    48 8b 05 d9 2f 00 00 mov    0x2fd9(%rip),%rax      # 3fe8
<__gmon_start__@Base>
  100f:    48 85 c0              test   %rax,%rax
  1012:    74 02                 je     1016 <_init+0x16>
  1014:    ff d0                 call   *%rax
  1016:    48 83 c4 08          add    $0x8,%rsp
  101a:    c3                    ret

```

Disassembly of section .plt:

```

00000000000001020 <.plt>:
   1020:    ff 35 8a 2f 00 00    push   0x2f8a(%rip)          # 3fb0
<_GLOBAL_OFFSET_TABLE_+0x8>
   1026:    ff 25 8c 2f 00 00    jmp     *0x2f8c(%rip)        # 3fb8
<_GLOBAL_OFFSET_TABLE_+0x10>
  102c:    0f 1f 40 00          nopl   0x0(%rax)
  1030:    f3 0f 1e fa          endbr64
  1034:    68 00 00 00 00      push   $0x0
  1039:    e9 e2 ff ff ff      jmp     1020 <_init+0x20>
  103e:    66 90                xchg   %ax,%ax

```

```

1040:      f3 0f 1e fa      endbr64
1044:      68 01 00 00 00    push    $0x1
1049:      e9 d2 ff ff ff    jmp     1020 <_init+0x20>
104e:      66 90             xchg    %ax,%ax
1050:      f3 0f 1e fa      endbr64
1054:      68 02 00 00 00    push    $0x2
1059:      e9 c2 ff ff ff    jmp     1020 <_init+0x20>
105e:      66 90             xchg    %ax,%ax

```

Disassembly of section .plt.got:

```

0000000000001060 <__cxa_finalize@plt>:
1060:      f3 0f 1e fa      endbr64
1064:      ff 25 8e 2f 00 00    jmp     *0x2f8e(%rip)          # 3ff8
<__cxa_finalize@GLIBC_2.2.5>
106a:      66 0f 1f 44 00 00    nopw    0x0(%rax,%rax,1)

```

Disassembly of section .plt.sec:

```

0000000000001070 <__stack_chk_fail@plt>:
1070:      f3 0f 1e fa      endbr64
1074:      ff 25 46 2f 00 00    jmp     *0x2f46(%rip)          # 3fc0
<__stack_chk_fail@GLIBC_2.4>
107a:      66 0f 1f 44 00 00    nopw    0x0(%rax,%rax,1)

```

```

0000000000001080 <printf@plt>:
1080:      f3 0f 1e fa      endbr64
1084:      ff 25 3e 2f 00 00    jmp     *0x2f3e(%rip)          # 3fc8
<printf@GLIBC_2.2.5>
108a:      66 0f 1f 44 00 00    nopw    0x0(%rax,%rax,1)

```

```

0000000000001090 <__isoc99_scanf@plt>:
1090:      f3 0f 1e fa      endbr64
1094:      ff 25 36 2f 00 00    jmp     *0x2f36(%rip)          # 3fd0
<__isoc99_scanf@GLIBC_2.7>
109a:      66 0f 1f 44 00 00    nopw    0x0(%rax,%rax,1)

```

Disassembly of section .text:

```

00000000000010a0 <_start>:
10a0:      f3 0f 1e fa      endbr64
10a4:      31 ed             xor     %ebp,%ebp
10a6:      49 89 d1          mov     %rdx,%r9
10a9:      5e                pop     %rsi
10aa:      48 89 e2          mov     %rsp,%rdx
10ad:      48 83 e4 f0       and     $0xfffffffffffffff0,%rsp
10b1:      50                push    %rax
10b2:      54                push    %rsp
10b3:      45 31 c0          xor     %r8d,%r8d
10b6:      31 c9             xor     %ecx,%ecx
10b8:      48 8d 3d ca 00 00 00    lea     0xca(%rip),%rdi          # 1189
<main>
10bf:      ff 15 13 2f 00 00    call    *0x2f13(%rip)          # 3fd8
<__libc_start_main@GLIBC_2.34>
10c5:      f4                hlt
10c6:      66 2e 0f 1f 84 00 00    cs nopw 0x0(%rax,%rax,1)

```

```

10cd:      00 00 00

00000000000010d0 <deregister_tm_clones>:
10d0:      48 8d 3d 39 2f 00 00    lea     0x2f39(%rip),%rdi      # 4010
<__TMC_END__>
10d7:      48 8d 05 32 2f 00 00    lea     0x2f32(%rip),%rax      # 4010
<__TMC_END__>
10de:      48 39 f8                cmp     %rdi,%rax
10e1:      74 15                je      10f8 <deregister_tm_clones+0x28>
10e3:      48 8b 05 f6 2e 00 00    mov     0x2ef6(%rip),%rax      # 3fe0
<__ITM_deregisterTMCloneTable@Base>
10ea:      48 85 c0                test    %rax,%rax
10ed:      74 09                je      10f8 <deregister_tm_clones+0x28>
10ef:      ff e0                jmp     *%rax
10f1:      0f 1f 80 00 00 00 00    nopl    0x0(%rax)
10f8:      c3                ret
10f9:      0f 1f 80 00 00 00 00    nopl    0x0(%rax)

0000000000001100 <register_tm_clones>:
1100:      48 8d 3d 09 2f 00 00    lea     0x2f09(%rip),%rdi      # 4010
<__TMC_END__>
1107:      48 8d 35 02 2f 00 00    lea     0x2f02(%rip),%rsi      # 4010
<__TMC_END__>
110e:      48 29 fe                sub     %rdi,%rsi
1111:      48 89 f0                mov     %rsi,%rax
1114:      48 c1 ee 3f                shr     $0x3f,%rsi
1118:      48 c1 f8 03                sar     $0x3,%rax
111c:      48 01 c6                add     %rax,%rsi
111f:      48 d1 fe                sar     $1,%rsi
1122:      74 14                je      1138 <register_tm_clones+0x38>
1124:      48 8b 05 c5 2e 00 00    mov     0x2ec5(%rip),%rax      # 3ff0
<__ITM_registerTMCloneTable@Base>
112b:      48 85 c0                test    %rax,%rax
112e:      74 08                je      1138 <register_tm_clones+0x38>
1130:      ff e0                jmp     *%rax
1132:      66 0f 1f 44 00 00        nopw    0x0(%rax,%rax,1)
1138:      c3                ret
1139:      0f 1f 80 00 00 00 00    nopl    0x0(%rax)

0000000000001140 <__do_global_ctors_aux>:
1140:      f3 0f 1e fa                endbr64
1144:      80 3d c5 2e 00 00 00    cmpb    $0x0,0x2ec5(%rip)      # 4010
<__TMC_END__>
114b:      75 2b                jne     1178 <__do_global_ctors_aux+0x38>
114d:      55                push    %rbp
114e:      48 83 3d a2 2e 00 00    cmpq    $0x0,0x2ea2(%rip)      # 3ff8
<__cxa_finalize@GLIBC_2.2.5>
1155:      00
1156:      48 89 e5                mov     %rsp,%rbp
1159:      74 0c                je      1167 <__do_global_ctors_aux+0x27>
115b:      48 8b 3d a6 2e 00 00    mov     0x2ea6(%rip),%rdi      # 4008
<__dso_handle>
1162:      e8 f9 fe ff ff                call    1060 <__cxa_finalize@plt>
1167:      e8 64 ff ff ff                call    10d0 <deregister_tm_clones>
116c:      c6 05 9d 2e 00 00 01    movb    $0x1,0x2e9d(%rip)      # 4010
<__TMC_END__>

```

```

1173:      5d                pop    %rbp
1174:      c3                ret
1175:      0f 1f 00          nopl   (%rax)
1178:      c3                ret
1179:      0f 1f 80 00 00 00 00  nopl   0x0(%rax)

0000000000001180 <frame_dummy>:
1180:      f3 0f 1e fa      endbr64
1184:      e9 77 ff ff      jmp    1100 <register_tm_clones>

0000000000001189 <main>:
1189:      f3 0f 1e fa      endbr64
118d:      55                push   %rbp
118e:      48 89 e5          mov    %rsp,%rbp
1191:      48 83 ec 10       sub    $0x10,%rsp
1195:      64 48 8b 04 25 28 00  mov    %fs:0x28,%rax
119c:      00 00
119e:      48 89 45 f8        mov    %rax,-0x8(%rbp)
11a2:      31 c0              xor    %eax,%eax
11a4:      48 8d 55 f4        lea    -0xc(%rbp),%rdx
11a8:      48 8d 45 f0        lea    -0x10(%rbp),%rax
11ac:      48 89 c6          mov    %rax,%rsi
11af:      48 8d 05 4e 0e 00 00  lea    0xe4e(%rip),%rax      # 2004
<_IO_stdin_used+0x4>
11b6:      48 89 c7          mov    %rax,%rdi
11b9:      b8 00 00 00 00     mov    $0x0,%eax
11be:      e8 cd fe ff ff     call   1090 <__isoc99_scanf@plt>
11c3:      8b 55 f0          mov    -0x10(%rbp),%edx
11c6:      8b 45 f4          mov    -0xc(%rbp),%eax
11c9:      01 d0          add    %edx,%eax
11cb:      89 c6          mov    %eax,%esi
11cd:      48 8d 05 36 0e 00 00  lea    0xe36(%rip),%rax      # 200a
<_IO_stdin_used+0xa>
11d4:      48 89 c7          mov    %rax,%rdi
11d7:      b8 00 00 00 00     mov    $0x0,%eax
11dc:      e8 9f fe ff ff     call   1080 <printf@plt>
11e1:      b8 00 00 00 00     mov    $0x0,%eax
11e6:      48 8b 55 f8        mov    -0x8(%rbp),%rdx
11ea:      64 48 2b 14 25 28 00  sub    %fs:0x28,%rdx
11f1:      00 00
11f3:      74 05          je     11fa <main+0x71>
11f5:      e8 76 fe ff ff     call   1070 <__stack_chk_fail@plt>
11fa:      c9          leave
11fb:      c3                ret

```

Disassembly of section .fini:

```

00000000000011fc <_fini>:
11fc:      f3 0f 1e fa      endbr64
1200:      48 83 ec 08      sub    $0x8,%rsp
1204:      48 83 c4 08      add    $0x8,%rsp
1208:      c3                ret

```

```
mips-linux-gnu-objdump -d adds.o > test
```

test文件的内容

adds.o: 文件格式 elf32-tradbigmips

Disassembly of section .text:

00000000 <main>:

0:	27bdfbfd0	addiu	sp,sp,-48
4:	afbf002c	sw	ra,44(sp)
8:	afbe0028	sw	s8,40(sp)
c:	03a0f025	move	s8,sp
10:	3c1c0000	lui	gp,0x0
14:	279c0000	addiu	gp,gp,0
18:	afbc0010	sw	gp,16(sp)
1c:	8f820000	lw	v0,0(gp)
20:	8c420000	lw	v0,0(v0)
24:	afc20024	sw	v0,36(s8)
28:	27c30020	addiu	v1,s8,32
2c:	27c2001c	addiu	v0,s8,28
30:	00603025	move	a2,v1
34:	00402825	move	a1,v0
38:	3c020000	lui	v0,0x0
3c:	24440000	addiu	a0,v0,0
40:	8f820000	lw	v0,0(gp)
44:	0040c825	move	t9,v0
48:	0320f809	jalr	t9
4c:	00000000	nop	
50:	8fdc0010	lw	gp,16(s8)
54:	8fc3001c	lw	v1,28(s8)
58:	8fc20020	lw	v0,32(s8)
5c:	00621021	addu	v0,v1,v0
60:	00402825	move	a1,v0
64:	3c020000	lui	v0,0x0
68:	24440008	addiu	a0,v0,8
6c:	8f820000	lw	v0,0(gp)
70:	0040c825	move	t9,v0
74:	0320f809	jalr	t9
78:	00000000	nop	
7c:	8fdc0010	lw	gp,16(s8)
80:	00001025	move	v0,zero
84:	00402025	move	a0,v0
88:	8f820000	lw	v0,0(gp)
8c:	8fc30024	lw	v1,36(s8)
90:	8c420000	lw	v0,0(v0)
94:	10620005	beq	v1,v0,ac <main+0xac>
98:	00000000	nop	
9c:	8f820000	lw	v0,0(gp)
a0:	0040c825	move	t9,v0
a4:	0320f809	jalr	t9
a8:	00000000	nop	
ac:	00801025	move	v0,a0
b0:	03c0e825	move	sp,s8
b4:	8fbf002c	lw	ra,44(sp)
b8:	8fbe0028	lw	s8,40(sp)

```
bc: 27bd0030      addiu  sp,sp,48
c0: 03e00008      jr      ra
c4: 00000000      nop
...
```

```
mips-linux-gnu-gcc -o adds.o adds.c
mips-linux-gnu-objdump -d adds.o > test
```

test文件的内容

adds.o: 文件格式 elf32-tradbigmips

Disassembly of section .init:

004005bc <_init>:

```
4005bc: 3c1c0002      lui      gp,0x2
4005c0: 279c7a54      addiu   gp,gp,31316
4005c4: 0399e021      addu    gp,gp,t9
4005c8: 27bdffe0      addiu   sp,sp,-32
4005cc: afbc0010      sw      gp,16(sp)
4005d0: afbf001c      sw      ra,28(sp)
4005d4: 8f828024      lw      v0,-32732(gp)
4005d8: 10400004      beqz    v0,4005ec <_init+0x30>
4005dc: 00000000      nop
4005e0: 8f998024      lw      t9,-32732(gp)
4005e4: 0320f809      jalr    t9
4005e8: 00000000      nop
4005ec: 8fbf001c      lw      ra,28(sp)
4005f0: 03e00008      jr      ra
4005f4: 27bd0020      addiu   sp,sp,32
```

Disassembly of section .text:

00400600 <__start>:

```
400600: 03e00025      move    zero,ra
400604: 04110001      bal     40060c <__start+0xc>
400608: 00000000      nop
40060c: 3c1c0002      lui      gp,0x2
400610: 279c7a04      addiu   gp,gp,31236
400614: 039fe021      addu    gp,gp,ra
400618: 0000f825      move    ra,zero
40061c: 8f848018      lw      a0,-32744(gp)
400620: 8fa50000      lw      a1,0(sp)
400624: 27a60004      addiu   a2,sp,4
400628: 2401fff8      li      at,-8
40062c: 03a1e824      and     sp,sp,at
400630: 27bdffe0      addiu   sp,sp,-32
400634: 00003825      move    a3,zero
400638: afa00010      sw      zero,16(sp)
40063c: afa20014      sw      v0,20(sp)
400640: afbd0018      sw      sp,24(sp)
400644: 8f998038      lw      t9,-32712(gp)
400648: 0320f809      jalr    t9
40064c: 00000000      nop
```

```

00400650 <hlt>:
    400650:    1000ffff        b      400650 <hlt>
    400654:    00000000        nop
        ...

00400660 <deregister_tm_clones>:
    400660:    3c040042        lui     a0,0x42
    400664:    3c020042        lui     v0,0x42
    400668:    24840014        addiu   a0,a0,20
    40066c:    24420014        addiu   v0,v0,20
    400670:    10440007        beq     v0,a0,400690 <deregister_tm_clones+0x30>
    400674:    3c1c0043        lui     gp,0x43
    400678:    279c8010        addiu   gp,gp,-32752
    40067c:    8f998034        lw      t9,-32716(gp)
    400680:    13200003        beqz    t9,400690 <deregister_tm_clones+0x30>
    400684:    00000000        nop
    400688:    03200008        jr      t9
    40068c:    00000000        nop
    400690:    03e00008        jr      ra
    400694:    00000000        nop

00400698 <register_tm_clones>:
    400698:    3c040042        lui     a0,0x42
    40069c:    3c020042        lui     v0,0x42
    4006a0:    24840014        addiu   a0,a0,20
    4006a4:    24450014        addiu   a1,v0,20
    4006a8:    00a42823        subu    a1,a1,a0
    4006ac:    00051083        sra     v0,a1,0x2
    4006b0:    00052fc2        srl     a1,a1,0x1f
    4006b4:    00a22821        addu    a1,a1,v0
    4006b8:    00052843        sra     a1,a1,0x1
    4006bc:    10a00007        beqz    a1,4006dc <register_tm_clones+0x44>
    4006c0:    3c1c0043        lui     gp,0x43
    4006c4:    279c8010        addiu   gp,gp,-32752
    4006c8:    8f99801c        lw      t9,-32740(gp)
    4006cc:    13200003        beqz    t9,4006dc <register_tm_clones+0x44>
    4006d0:    00000000        nop
    4006d4:    03200008        jr      t9
    4006d8:    00000000        nop
    4006dc:    03e00008        jr      ra
    4006e0:    00000000        nop

004006e4 <__do_global_dtors_aux>:
    4006e4:    27bdf0e0        addiu   sp,sp,-32
    4006e8:    afb00018        sw      s0,24(sp)
    4006ec:    3c100042        lui     s0,0x42
    4006f0:    afbf001c        sw      ra,28(sp)
    4006f4:    92020050        lbu     v0,80(s0)
    4006f8:    14400006        bnez    v0,400714 <__do_global_dtors_aux+0x30>
    4006fc:    8fbf001c        lw      ra,28(sp)
    400700:    0c100198        jal     400660 <deregister_tm_clones>
    400704:    00000000        nop
    400708:    24020001        li      v0,1
    40070c:    a2020050        sb      v0,80(s0)
    400710:    8fbf001c        lw      ra,28(sp)
    400714:    8fb00018        lw      s0,24(sp)

```

```

400718:      03e00008      jr      ra
40071c:      27bd0020      addiu   sp,sp,32

00400720 <frame_dummy>:
400720:      081001a6      j       400698 <register_tm_clones>
400724:      00000000      nop
...

00400730 <main>:
400730:      27bdffd0      addiu   sp,sp,-48
400734:      afbf002c      sw      ra,44(sp)
400738:      afbe0028      sw      s8,40(sp)
40073c:      03a0f025      move    s8,sp
400740:      3c1c0043      lui     gp,0x43
400744:      279c8010      addiu   gp,gp,-32752
400748:      afbc0010      sw      gp,16(sp)
40074c:      8f82802c      lw      v0,-32724(gp)
400750:      8c420000      lw      v0,0(v0)
400754:      afc20024      sw      v0,36(s8)
400758:      27c30020      addiu   v1,s8,32
40075c:      27c2001c      addiu   v0,s8,28
400760:      00603025      move    a2,v1
400764:      00402825      move    a1,v0
400768:      3c020040      lui     v0,0x40
40076c:      24440890      addiu   a0,v0,2192
400770:      8f828020      lw      v0,-32736(gp)
400774:      0040c825      move    t9,v0
400778:      0320f809      jalr    t9
40077c:      00000000      nop
400780:      8fdc0010      lw      gp,16(s8)
400784:      8fc3001c      lw      v1,28(s8)
400788:      8fc20020      lw      v0,32(s8)
40078c:      00621021      addu    v0,v1,v0
400790:      00402825      move    a1,v0
400794:      3c020040      lui     v0,0x40
400798:      24440898      addiu   a0,v0,2200
40079c:      8f828030      lw      v0,-32720(gp)
4007a0:      0040c825      move    t9,v0
4007a4:      0320f809      jalr    t9
4007a8:      00000000      nop
4007ac:      8fdc0010      lw      gp,16(s8)
4007b0:      00001025      move    v0,zero
4007b4:      00402025      move    a0,v0
4007b8:      8f82802c      lw      v0,-32724(gp)
4007bc:      8fc30024      lw      v1,36(s8)
4007c0:      8c420000      lw      v0,0(v0)
4007c4:      10620005      beq     v1,v0,4007dc <main+0xac>
4007c8:      00000000      nop
4007cc:      8f828028      lw      v0,-32728(gp)
4007d0:      0040c825      move    t9,v0
4007d4:      0320f809      jalr    t9
4007d8:      00000000      nop
4007dc:      00801025      move    v0,a0
4007e0:      03c0e825      move    sp,s8
4007e4:      8fbf002c      lw      ra,44(sp)
4007e8:      8fbe0028      lw      s8,40(sp)

```

```

4007ec:      27bd0030      addiu   sp,sp,48
4007f0:      03e00008      jr      ra
4007f4:      00000000      nop
...

```

Disassembly of section `.MIPS.stubs`:

```

00400800 <_MIPS_STUBS_>:
400800:      8f998010      lw      t9,-32752(gp)
400804:      03e07825      move    t7,ra
400808:      0320f809      jalr    t9
40080c:      2418000c      li      t8,12
400810:      8f998010      lw      t9,-32752(gp)
400814:      03e07825      move    t7,ra
400818:      0320f809      jalr    t9
40081c:      2418000a      li      t8,10
400820:      8f998010      lw      t9,-32752(gp)
400824:      03e07825      move    t7,ra
400828:      0320f809      jalr    t9
40082c:      24180008      li      t8,8
400830:      8f998010      lw      t9,-32752(gp)
400834:      03e07825      move    t7,ra
400838:      0320f809      jalr    t9
40083c:      24180006      li      t8,6
...

```

Disassembly of section `.fini`:

```

00400850 <_fini>:
400850:      3c1c0002      lui     gp,0x2
400854:      279c77c0      addiu   gp,gp,30656
400858:      0399e021      addu    gp,gp,t9
40085c:      27bdffe0      addiu   sp,sp,-32
400860:      afbc0010      sw      gp,16(sp)
400864:      afbf001c      sw      ra,28(sp)
400868:      8fbf001c      lw      ra,28(sp)
40086c:      03e00008      jr      ra
400870:      27bd0020      addiu   sp,sp,32

```

2、readelf 程序的问题

- 尝试使用我们编写的readelf程序，解析之前在target目录下生成的内核ELF文件。

```

# ~/23371325/tools/readelf目录下
make
./readelf ~/23371325/target/mos

```

```

0:0x0
1:0x80020000
2:0x80021900
3:0x80021918
4:0x80021930
5:0x0
6:0x0
7:0x0

```

```
8:0x0
9:0x0
10:0x0
11:0x0
12:0x0
13:0x0
14:0x0
15:0x0
16:0x0
17:0x0
```

- 也许你会发现我们编写的readelf程序是不能解析readelf文件本身的，而我们刚才介绍的系统工具readelf则可以解析，这是为什么呢？（提示：尝试使用readelf-h，并阅读tools/readelf目录下的Makefile，观察readelf与hello的不同）

```
# ~/23371325/tools/readelf目录下
readelf -h ./readelf
```

```
ELF 头:
Magic:      7f 45 4c 46 02 01 01 00 00 00 00 00 00 00 00
类别:                               ELF64
数据:                               2 补码, 小端序 (little endian)
Version:    1 (current)
OS/ABI:      UNIX - System V
ABI 版本:    0
类型:        DYN (Position-Independent Executable file)
系统架构:    Advanced Micro Devices x86-64
版本:        0x1
入口点地址:      0x1180
程序头起点:      64 (bytes into file)
Start of section headers: 14488 (bytes into file)
标志:          0x0
Size of this header: 64 (bytes)
Size of program headers: 56 (bytes)
Number of program headers: 13
Size of section headers: 64 (bytes)
Number of section headers: 31
Section header string table index: 30
```

```
# ~/23371325/tools/readelf目录下
readelf -h hello
```

```
ELF 头:
Magic:      7f 45 4c 46 01 01 01 03 00 00 00 00 00 00 00
类别:                               ELF32
数据:                               2 补码, 小端序 (little endian)
Version:    1 (current)
OS/ABI:      UNIX - GNU
ABI 版本:    0
类型:        EXEC (可执行文件)
系统架构:    Intel 80386
版本:        0x1
入口点地址:      0x8049750
程序头起点:      52 (bytes into file)
```

```
Start of section headers:          707128 (bytes into file)
标志:                             0x0
Size of this header:               52 (bytes)
Size of program headers:          32 (bytes)
Number of program headers:         8
Size of section headers:          40 (bytes)
Number of section headers:         30
Section header string table index: 29
```

```
# Makefile的内容
%.o: %.c
    $(CC) -c $<

.PHONY: clean

readelf: main.o readelf.o
    $(CC) $^ -o $@

hello: hello.c
    $(CC) $^ -o $@ -m32 -static -g

clean:
    rm -f *.o readelf hello
```

通过观察可以发现生成的hello文件的类别为32ELF，在Makefile中也确实是用 `-m32` 生成的，而 readelf文件的类别为64ELF，所以readelf无法解析自身

3、内核入口地址的问题

- 在理论课上我们了解到，MIPS体系结构上电时，启动入口地址为0xBFC00000（其实启动入口地址是根据具体型号而定的，由硬件逻辑确定，也有可能不是这个地址，但一定是一个确定的地址），但实验操作系统的内核入口并没有放在上电启动地址，而是按照内存布局图放置。思考为什么这样放置内核还能保证内核入口被正确跳转到？（提示：思考实验中启动过程的两阶段分别由谁执行。）

在 MIPS 体系结构中，上电时 CPU 会从固定的地址（如 0xBFC00000）开始执行代码。这个地址通常映射到 Boot ROM 或 Flash，其中存储了 Bootloader。而在实验中，我们已经设置好了各个节被加载的位置，即最终的segment地址，同时在kernel.ld中使用了ENTRY(_start)指定了程序入口（即内核入口），最终保证了内核入口能够被正确跳转。

二、难点分析

本次实验第一个难点我认为是理解ELF文件的结构并补全readelf.c文件。我们需要先理解elf.h头文件的结构体，在补全代码的时候需要结合该头文件进行对应的类型转换以保证程序能正常执行，要将binary和sh_table这两个地址转为char型指针

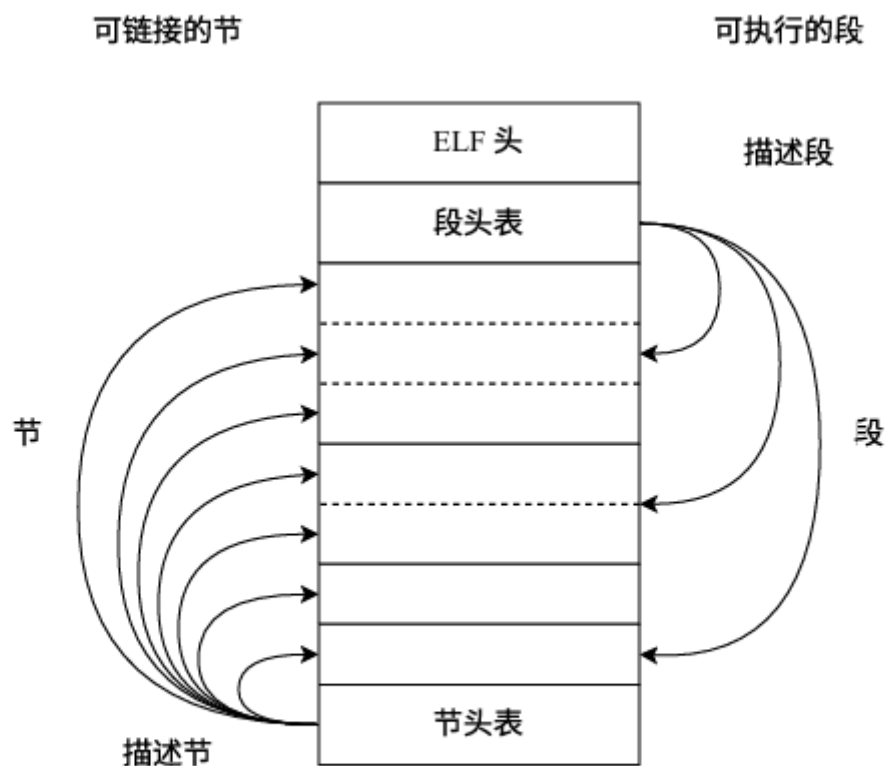


图 1.1: ELF 文件结构

本次实验第二个难点就是根据内存布局图补全kernel.ld和start.S两个文件

本次实验最后一个难点就是补全print.c，虽然每个待补全部分都比较简单，但八个待补全部分也增加了犯错的可能，为确保最后实现正确需要使用gdb多加调试来排除错误

三、实验体会

本次实验相较于上一次实验对一些基础操作的练习，正式地开始了操作系统的搭建——从解析elf文件到加载内核，到最后的实现printf函数。这一次实验的文件也远远多于上一次，需要阅读和理解的代码量也比上一次更加庞大，很多时候面对着冰冷的vim界面完全两眼抓瞎不知从何处下手。所以我将一些文件的内容传给大语言模型，同时结合指导书，一点点理解了这次实验的内容。希望可以继续稳扎稳打，攻克未来的挑战！

四、原创说明

- 本文部分内容在LLM（大语言模型）的协助下完成